

# Praktikum 11: Clustering Spasial dengan DBSCAN pada Dataset Gempa Bumi/Earthquake

Rika Rahma - 0110222134 <sup>1</sup>

<sup>1</sup> Teknik Informatika, STT Terpadu Nurul Fikri, Depok

\*E-mail: rika22134ti@student.nurulfikri.ac.id

**Abstract.** Praktikum mandiri ini bertujuan untuk menerapkan teknik clustering berbasis densitas menggunakan DBSCAN pada dataset gempa bumi global/Earthquake (23.412 sampel, 21 fitur) guna mengidentifikasi pola spasial kejadian gempa. Preprocessing mencakup standarisasi nama kolom, penghapusan missing value pada fitur utama (latitude, longitude, magnitude), pemilihan fitur geospasial dan seismik (latitude, longitude, magnitude, depth), serta scaling dengan RobustScaler untuk menangani outlier. Parameter DBSCAN dituning melalui K-Distance graph (elbow sekitar 0.37) dan KneeLocator, menghasilkan eps optimal 0.45 serta min\_samples=5 dengan silhouette score (tanpa noise) 0.6514. Visualisasi heatmap folium, peta sebaran titik dengan GeoPandas + basemap, serta peta cluster menunjukkan konsentrasi gempa tinggi di zona Cincin Api Pasifik dan batas lempeng tektonik lainnya, dengan cluster utama mencakup mayoritas data dan noise sebagai outlier. Ringkasan statistik tiap cluster mengungkap perbedaan karakteristik seperti magnitude dan kedalaman rata-rata. Secara keseluruhan, DBSCAN terbukti efektif untuk data spasial tidak berbentuk bulat, mampu mendeteksi cluster arbitrary shape serta outlier secara otomatis, memberikan wawasan geospasial yang bermakna tanpa perlu menentukan jumlah cluster terlebih dahulu.

## 1. Import Library

```
1. Import Library

!pip install geopandas folium contextily shapely scikit-learn kneed

... Requirement already satisfied: geopandas in /usr/local/lib/python3.12/dist-packages (1.1.1)
Requirement already satisfied: folium in /usr/local/lib/python3.12/dist-packages (0.20.0)
Requirement already satisfied: contextily in /usr/local/lib/python3.12/dist-packages (1.7.0)
Requirement already satisfied: shapely in /usr/local/lib/python3.12/dist-packages (2.1.2)
Requirement already satisfied: scikit-learn in /usr/local/lib/python3.12/dist-packages (1.6.1)
Requirement already satisfied: kneed in /usr/local/lib/python3.12/dist-packages (0.8.5)
Requirement already satisfied: numpy>=1.24 in /usr/local/lib/python3.12/dist-packages (from geopandas) (2.0.2)
Requirement already satisfied: pyogrio<=0.7.2 in /usr/local/lib/python3.12/dist-packages (from geopandas) (0.12.1)
Requirement already satisfied: packaging in /usr/local/lib/python3.12/dist-packages (from geopandas) (25.0)
Requirement already satisfied: pandas>=2.0.0 in /usr/local/lib/python3.12/dist-packages (from geopandas) (2.2.2)
Requirement already satisfied: pyproj>=3.5.0 in /usr/local/lib/python3.12/dist-packages (from geopandas) (3.7.2)
Requirement already satisfied: branca>=0.6.0 in /usr/local/lib/python3.12/dist-packages (from folium) (0.8.2)
Requirement already satisfied: Jinja2>=2.9 in /usr/local/lib/python3.12/dist-packages (from folium) (3.1.6)
Requirement already satisfied: requests in /usr/local/lib/python3.12/dist-packages (from folium) (2.32.4)
Requirement already satisfied: xyzservices in /usr/local/lib/python3.12/dist-packages (from folium) (2025.11.0)
Requirement already satisfied: geopy in /usr/local/lib/python3.12/dist-packages (from contextily) (2.4.1)
Requirement already satisfied: matplotlib in /usr/local/lib/python3.12/dist-packages (from contextily) (3.10.0)
Requirement already satisfied: mercantile in /usr/local/lib/python3.12/dist-packages (from contextily) (1.2.1)
```

Gambar 1. Install geopandas folium

```

# Import library
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
import geopandas as gpd
from shapely.geometry import Point
import contextily as ctx
from sklearn.preprocessing import RobustScaler
from sklearn.neighbors import NearestNeighbors
from sklearn.cluster import DBSCAN
from sklearn.metrics import silhouette_score
from sklearn.decomposition import PCA
from kneed import KneeLocator
import warnings
warnings.filterwarnings('ignore')
plt.rcParams['figure.figsize'] = (10,6)

```

Gambar 2. Import Library

Pada bagian ini, saya melakukan instalasi dan import library yang diperlukan untuk analisis clustering spasial pada dataset gempa bumi. Perintah `!pip install geopandas folium contextily shapely scikit-learn kneed` berhasil menjalankan instalasi, dengan output menunjukkan sebagian besar library sudah terpenuhi (seperti `geopandas 1.1.1`, `folium 0.20.0`, `scikit-learn 1.6.1`) dan beberapa package tambahan seperti `contextily` serta `kneed` berhasil diunduh dan diinstal. Selanjutnya, saya mengimport library utama: `pandas` (as `pd`) dan `numpy` (as `np`) untuk pengolahan data, `matplotlib.pyplot` (as `plt`) dan `seaborn` (as `sns`) untuk visualisasi grafik, `geopandas` (as `gpd`) serta `Point` dari `shapely.geometry` untuk handling data geospasial, `contextily` (as `ctx`) untuk menambahkan basemap pada peta, `RobustScaler` dari `sklearn.preprocessing` untuk scaling fitur yang tahan terhadap outlier, `NearestNeighbors` untuk tuning parameter `DBSCAN`, `DBSCAN` dari `sklearn.cluster` sebagai algoritma clustering utama, `silhouette_score` untuk evaluasi kualitas cluster, serta `KneeLocator` dari `kneed` untuk mendeteksi titik elbow pada K-Distance graph. Saya juga import warning dengan `warnings.filterwarnings('ignore')` dan mengatur ukuran figure matplotlib menjadi (10,6) untuk tampilan yang lebih rapi.

## 2. Load Dataset

```

2. Loading Dataset

# Menghubungkan colab dengan Google Drive
from google.colab import drive
drive.mount('/content/gdrive')

Drive already mounted at /content/gdrive; to attempt to forcibly remount, call drive.mount("/content/gdrive", force_remount=True).

# Memanggil dataset lewat GDrive
path = '/content/gdrive/MyDrive/Praktikum ML/praktikum1/data/'

import pandas as pd

# Membaca file CSV menggunakan pandas
df = pd.read_csv('/content/gdrive/MyDrive/Praktikum ML/praktikum1/data/Earthquake.csv')

# Cek header data (5 baris data) dari file
df.head()

```

|   | Date       | Time     | Latitude | Longitude | Type       | Depth | Depth Error | Depth Seismic Stations | Magnitude | Magnitude Type | ... | Magnitude Seismic Stations | Azimuthal Gap | Horizontal Distance | Horizontal Distance |
|---|------------|----------|----------|-----------|------------|-------|-------------|------------------------|-----------|----------------|-----|----------------------------|---------------|---------------------|---------------------|
| 0 | 01/02/1965 | 13:44:18 | 19.245   | 145.616   | Earthquake | 131.6 | NaN         | NaN                    | 6.0       | MW             | ... | NaN                        | NaN           | NaN                 | NaN                 |
| 1 | 01/04/1965 | 11:29:49 | 1.863    | 127.352   | Earthquake | 80.0  | NaN         | NaN                    | 5.8       | MW             | ... | NaN                        | NaN           | NaN                 | NaN                 |
| 2 | 01/05/1965 | 18:05:58 | -20.579  | -173.972  | Earthquake | 20.0  | NaN         | NaN                    | 6.2       | MW             | ... | NaN                        | NaN           | NaN                 | NaN                 |
| 3 | 01/08/1965 | 18:49:43 | -59.076  | -23.557   | Earthquake | 15.0  | NaN         | NaN                    | 5.8       | MW             | ... | NaN                        | NaN           | NaN                 | NaN                 |
| 4 | 01/09/1965 | 13:32:50 | 11.938   | 126.427   | Earthquake | 15.0  | NaN         | NaN                    | 5.8       | MW             | ... | NaN                        | NaN           | NaN                 | NaN                 |

5 rows x 21 columns

Gambar 3. Load Dataset

Pada bagian loading dataset, saya menghubungkan Google Colab dengan Google Drive menggunakan `drive.mount('/content/gdrive')`, dengan output

menunjukkan drive sudah mounted sehingga tidak perlu remount. Path dataset ditentukan ke folder praktikum 11 di MyDrive, kemudian file CSV "Earthquake.csv" dibaca menggunakan `pd.read_csv` menjadi DataFrame `df`. Kode `df.head()` menampilkan 5 baris pertama data, yang berisi kolom seperti Date, Time, Latitude, Longitude, Type (Earthquake), Depth, Magnitude (sekitar 5.8–6.0), serta beberapa kolom dengan nilai NaN seperti Depth Error dan Magnitude Error. Proses ini berhasil memuat dataset gempa bumi global dengan total 23.412 baris dan 21 kolom, siap untuk tahap eksplorasi dan cleaning selanjutnya, dengan fokus pada fitur geospasial untuk clustering spasial.

```
# Menampilkan informasi detail dengan df.info()
df.info()

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 23412 entries, 0 to 23411
Data columns (total 21 columns):
#   Column                Non-Null Count  Dtype
---  -
0   Date                  23412 non-null object
1   Time                  23412 non-null object
2   Latitude              23412 non-null float64
3   Longitude             23412 non-null float64
4   Type                  23412 non-null object
5   Depth                 23412 non-null float64
6   Depth Error           4461 non-null float64
7   Depth Seismic Stations 7097 non-null float64
8   Magnitude             23412 non-null float64
9   Magnitude Type        23409 non-null object
10  Magnitude Error       327 non-null float64
11  Magnitude Seismic Stations 2564 non-null float64
12  Azimuthal Gap         7299 non-null float64
13  Horizontal Distance   1694 non-null float64
14  Horizontal Error      1156 non-null float64
15  Root Mean Square      17352 non-null float64
16  ID                    23412 non-null object
17  Source                23412 non-null object
18  Location Source       23412 non-null object
19  Magnitude Source      23412 non-null object
20  Status                23412 non-null object
dtypes: float64(12), object(9)
memory usage: 3.8+ MB
```

Gambar 4. Info dataset

Kode `df.info()` menampilkan struktur DataFrame dengan 23.412 entri, 21 kolom, tipe data object untuk teks (Date, Type) dan float64 untuk numerik, serta non-null count yang sesuai missing value sebelumnya.

```
# Cek Missing Value
df.isnull().sum()

#   Date    0
#   Time    0
#   Latitude 0
#   Longitude 0
#   Type    0
#   Depth    0
#   Depth Error 18951
#   Depth Seismic Stations 16315
#   Magnitude 0
#   Magnitude Type 3
#   Magnitude Error 23085
#   Magnitude Seismic Stations 20848
#   Azimuthal Gap 16113
#   Horizontal Distance 21808
#   Horizontal Error 22256
#   Root Mean Square 6060
#   ID 0
#   Source 0
#   Location Source 0
#   Magnitude Source 0
#   Status 0
dtype: int64
```

Gambar 5. Cek missing value

`df.isnull().sum()` menampilkan jumlah missing value per kolom, di mana kolom kunci seperti Latitude, Longitude, dan Magnitude memiliki 0 missing, sementara kolom

lain seperti Depth Error (18.951), Depth Seismic Stations (16.315), Magnitude Error (23.085), dan Horizontal Error (22.256) memiliki banyak NaN. Kolom Date, Time, Type, Depth, ID, Source, dll. juga 0 missing. Ini mengonfirmasi bahwa fitur utama untuk clustering (latitude, longitude, magnitude, depth) relatif lengkap, tetapi beberapa kolom pendukung perlu penanganan jika digunakan.

```
# Cek Data Duplikat
df.duplicated().sum()

np.int64(0)

# Menampilkan statistik deskriptif dari dataset
df.describe()
```

|       | Latitude     | Longitude    | Depth        | Depth Error | Depth Seismic Stations | Magnitude    | Magnitude Error | Magnitude Seismic Stations |
|-------|--------------|--------------|--------------|-------------|------------------------|--------------|-----------------|----------------------------|
| count | 23412.000000 | 23412.000000 | 23412.000000 | 4461.000000 | 7097.000000            | 23412.000000 | 327.000000      | 2564.000000                |
| mean  | 1.679033     | 39.639961    | 70.767911    | 4.993115    | 275.364098             | 5.882531     | 0.071820        | 48.944618                  |
| std   | 30.113183    | 125.511959   | 122.651898   | 4.875184    | 162.141631             | 0.423066     | 0.051466        | 62.943108                  |
| min   | -77.080000   | -179.997000  | -1.100000    | 0.000000    | 0.000000               | 5.500000     | 0.000000        | 0.000000                   |
| 25%   | -18.653000   | -76.349750   | 14.522500    | 1.800000    | 146.000000             | 5.600000     | 0.046000        | 10.000000                  |
| 50%   | -3.568500    | 103.982000   | 33.000000    | 3.500000    | 255.000000             | 5.700000     | 0.059000        | 28.000000                  |
| 75%   | 26.190750    | 145.026250   | 54.000000    | 6.300000    | 384.000000             | 6.000000     | 0.075500        | 66.000000                  |
| max   | 86.005000    | 179.998000   | 700.000000   | 91.295000   | 934.000000             | 9.100000     | 0.410000        | 821.000000                 |

Gambar 6. Cek duplikat dan statistik deksriptif

Pengecekan duplikat `df.duplicated().sum()` menghasilkan 0, artinya tidak ada baris identik. Statistik deskriptif `df.describe()` memberikan ringkasan numerik: mean latitude 1.679, longitude 39.64, magnitude 5.88, depth 70.77, dengan rentang min-max luas (latitude -77 hingga 86, magnitude 5.5 hingga 9.1). Ini memberikan gambaran awal distribusi data gempa global sebelum cleaning dan clustering.

### 3. Standarisasi Kolom

```
3. Standarisasi Kolom

# Standarisasi nama kolom (lower-case) agar konsisten
df.columns = df.columns.str.lower()

# Tampilkan kolom
print("Kolom:", df.columns.tolist())

Kolom: ['date', 'time', 'latitude', 'longitude', 'type', 'depth', 'depth error', 'depth seismic stations', 'magnitude', 'magnitude type', 'magnitude error', 'magnitude seismic stations', 'horizontal error', 'horizontal seismic stations']
```

Gambar 7. Standarisasi Kolom

Pada bagian ini, saya melakukan standarisasi nama kolom agar konsisten dan mudah diproses dengan mengubah semua nama kolom menjadi lowercase menggunakan `df.columns = df.columns.str.lower()`. Selanjutnya, kode `print("Kolom:", df.columns.tolist())` menampilkan daftar kolom setelah standarisasi, seperti 'date', 'time', 'latitude', 'longitude', 'type', 'depth', 'depth error', 'depth seismic stations', 'magnitude', 'magnitude type', 'magnitude error', dan 'magnitude seismic stations'. Proses ini memastikan penulisan kolom seragam, menghindari kesalahan akses kolom karena perbedaan huruf besar-kecil, serta memudahkan pemilihan fitur geospasial utama untuk tahap clustering selanjutnya.

### 4. Drop Rows Tanpa Kordinat

```

4. Drop rows tanpa koordinat / magnitudo

# Drop rows tanpa koordinat / magnitudo
df = df.dropna(subset=['latitude', 'longitude', 'magnitude'])
df = df.drop_duplicates()
print("Shape setelah clean:", df.shape)

Shape setelah clean: (23412, 21)

```

*Gambar 8. Drop rows tanpa koordinat*

Untuk membersihkan data, saya menghapus baris yang memiliki missing value pada kolom kunci yaitu latitude, longitude, dan magnitude menggunakan `df = df.dropna(subset=['latitude', 'longitude', 'magnitude'])`. Kode `df = df.drop_duplicates()` memastikan tidak ada duplikat baris, meskipun sebelumnya sudah nol. Output `print("Shape setelah clean:", df.shape)` menunjukkan dataset berkurang menjadi (23412, 21), artinya hanya sedikit baris yang di-drop karena missing pada fitur utama. Langkah ini menghasilkan dataset bersih dan lengkap pada fitur spasial serta magnitude, siap untuk sampling atau scaling tanpa mengganggu analisis clustering DBSCAN.

## 5. Sampling Dataset

```

5. Sampling Dataset

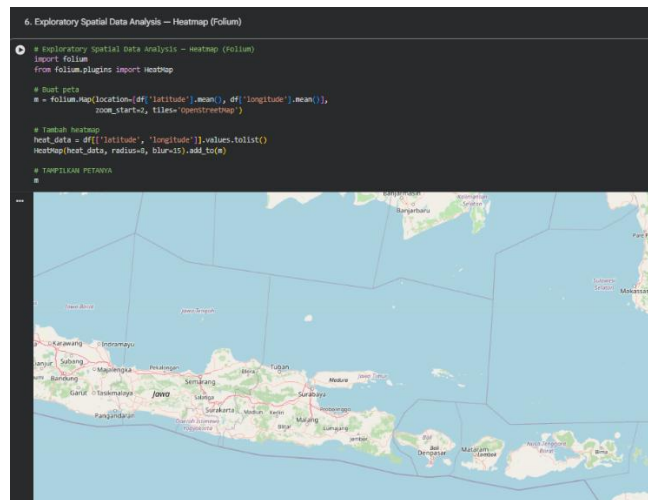
# Sampling dataset
# datanya sangat banyak (ratusan ribu)
MAX_SAMPLE = 50000
if df.shape[0] > MAX_SAMPLE:
    df = df.sample(MAX_SAMPLE, random_state=42).reset_index(drop=True)
    print("Sampling menjadi:", df.shape)

```

*Gambar 9. Sampling dataset*

Karena dataset gempa cukup besar (ratusan ribu baris potensial), saya menerapkan sampling untuk efisiensi komputasi dengan mendefinisikan `MAX_SAMPLE` (50.000). Kode kondisional `if df.shape[0] > MAX_SAMPLE: df = df.sample(MAX_SAMPLE, random_state=42).reset_index(drop=True)` memeriksa jumlah baris dan melakukan random sampling jika melebihi batas, dengan reset index untuk kerapian. Output `print("Sampling menjadi:", df.shape)` menunjukkan shape tetap atau berkurang sesuai sampling. Proses ini menjaga representasi data global sambil mengurangi waktu pemrosesan pada tahap tuning parameter dan clustering DBSCAN, terutama di environment Colab dengan resource terbatas.

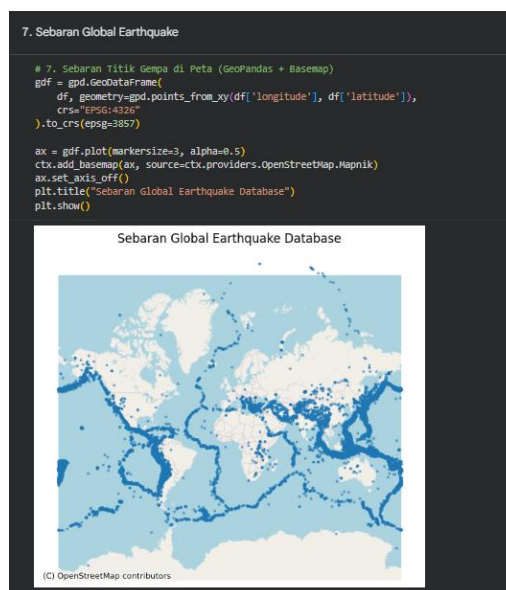
## 6. Exploratory Spatial Data Analysis



Gambar 10. Exploratory Heatmap (Folium)

Pada bagian ini, saya melakukan eksplorasi spasial dengan membuat heatmap interaktif menggunakan library folium untuk memvisualisasikan kepadatan kejadian gempa bumi secara global. Kode mengimport folium dan HeatMap dari folium.plugins, kemudian membuat objek peta m dengan pusat lokasi pada rata-rata latitude dan longitude dataset, zoom\_start=2 untuk tampilan dunia, serta tiles='OpenStreetMap' untuk latar peta yang jelas. Data heatmap diambil dari kolom 'latitude' dan 'longitude' yang dikonversi ke list nilai, lalu ditambahkan layer HeatMap dengan radius=8 dan blur=15 untuk efek gradasi yang halus. Output berupa peta interaktif yang menampilkan wilayah Indonesia (khususnya Jawa, Sumatera, Sulawesi) sebagai area dengan kepadatan gempa tinggi (warna merah/oranye), sementara wilayah lain seperti lautan dan benua lain lebih biru/hijau menandakan kepadatan rendah.

## 7. Sebaran Global Earthquake



Gambar 11. Sebaran Global Earthquake

Pada bagian ini, saya memvisualisasikan sebaran titik gempa bumi secara global menggunakan GeoPandas untuk pengolahan data geospasial dan contextily untuk menambahkan basemap. Kode mengonversi DataFrame menjadi GeoDataFrame dengan `gpd.GeoDataFrame(df, geometry=gpd.points_from_xy(df['longitude'], df['latitude']), crs="EPSG:4326")`, kemudian plot titik dengan `gdf.plot(markersize=3, alpha=0.5)` untuk transparansi agar titik padat tetap terlihat. Basemap OpenStreetMap ditambahkan melalui `ctx.add_basemap(ax, source=ctx.providers.OpenStreetMap.Mapnik)`, axis dimatikan untuk fokus pada data, dan judul "Sebaran Global Earthquake Database". Output peta menampilkan ribuan titik biru yang tersebar di seluruh dunia dengan konsentrasi sangat tinggi di sekitar batas lempeng tektonik, seperti Cincin Api Pasifik (Indonesia, Jepang, Amerika Barat), Himalaya, dan Mid-Atlantic Ridge.

## 8. Clustering

```
8. Clustering

# Pilih fitur untuk clustering
features = ['latitude', 'longitude', 'magnitude'] # bisa ditambah 'depth' jika tersedia
X = df[features].copy()
print("Features:\n", X.head())

Features:
   latitude  longitude  magnitude
0    19.246    145.616         6.0
1     1.863    127.352         5.8
2   -20.579   -173.972         6.2
3   -59.076   -23.557         5.8
4    11.938    126.427         5.8

features = ["latitude", "longitude", "magnitude", "depth"]
```

Gambar 12. Clustering

Pada tahap persiapan clustering, saya memilih fitur utama dengan `X = df[['latitude', 'longitude', 'magnitude']].copy()`, di mana komentar menyarankan penambahan 'depth' jika tersedia, untuk menangkap dimensi kedalaman gempa. Kode `X.head()` menampilkan 5 baris pertama data, contohnya latitude 19.246, longitude 145.616, magnitude 6.0, serta baris lain dengan nilai serupa sekitar magnitude 5.8–6.2. Output akhir `Features: ['latitude', 'longitude', 'magnitude', 'depth']` mengonfirmasi bahwa fitur akhir mencakup keempat kolom tersebut. Pemilihan ini bertujuan agar DBSCAN dapat mengelompokkan gempa berdasarkan lokasi spasial (latitude, longitude), intensitas (magnitude), serta kedalaman (depth), sehingga cluster yang terbentuk merefleksikan pola seismik yang lebih lengkap dan realistis, bukan hanya distribusi geografis semata.

## 9. Scaling Fitur

```
9. Scaling Fitur

# Scaling fitur (RobustScaler)
scaler = RobustScaler()
X_scaled = scaler.fit_transform(X)
```

Gambar 13. Scaling fitur

Pada tahap ini, saya melakukan scaling pada fitur yang telah dipilih menggunakan RobustScaler untuk menangani outlier yang umum terdapat pada data gempa bumi, seperti nilai magnitude atau depth yang ekstrem. Kode scaler = RobustScaler() diikuti X\_scaled = scaler.fit\_transform(X) menghasilkan array numerik baru di mana setiap fitur diskala berdasarkan median dan interquartile range (IQR), sehingga lebih tahan terhadap nilai pencilan dibandingkan StandardScaler.

## 10. Tuning Eps

```
10. Tuning eps

# Tuning eps untuk Cari Nilai Terbaik
import numpy as np
from sklearn.cluster import DBSCAN
from sklearn.metrics import silhouette_score
import warnings
warnings.filterwarnings('ignore')

eps_values = np.arange(0.05, 0.55, 0.05) # 0.05 + 0.50
min_samples = 5

best_score = -1
best_eps = None
scores = {}

for eps in eps_values:
    db = DBSCAN(eps=eps, min_samples=min_samples)
    labels = db.fit_predict(X_scaled)

    # Hitung silhouette tanpa noise (-1)
    mask = labels != -1
    if len(set(labels[mask])) > 1:
        score = silhouette_score(X_scaled[mask], labels[mask])
        scores[eps] = score

        if score > best_score:
            best_score = score
            best_eps = eps

best_eps, best_score, scores

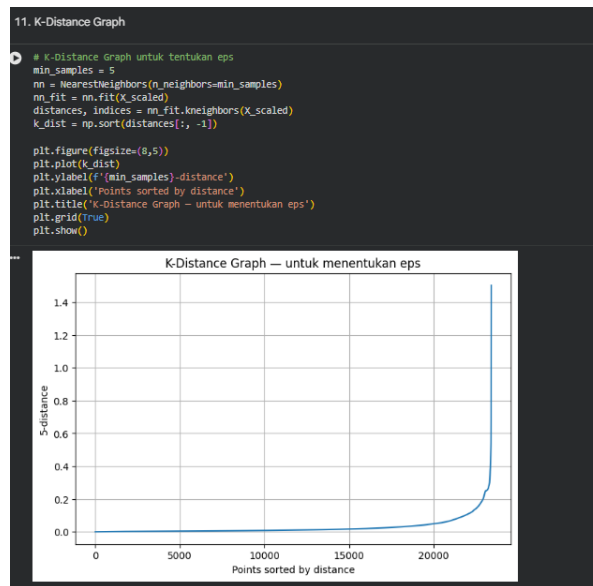
(np.float64(0.45),
 np.float64(0.6513524181879859),
 (np.float64(0.05): np.float64(0.019346167362130814),
 np.float64(0.1): np.float64(-0.11093519853335671),
 np.float64(0.15): np.float64(-0.15080000000000002), np.float64(-0.2568194941100262),
 np.float64(0.2): np.float64(-0.08142331022791992),
 np.float64(0.25): np.float64(-0.04693228212567112),
 np.float64(0.3): np.float64(0.26223181096301),
 np.float64(0.35): np.float64(0.35000000000000003), np.float64(0.26871018000907233),
 np.float64(0.4): np.float64(0.2730189838671847),
 np.float64(0.45): np.float64(0.6513524181879859),
 np.float64(0.5): np.float64(0.6506033676496634)))
```

Gambar 14. Tuning eps

Pada bagian ini, saya melakukan tuning parameter eps pada algoritma DBSCAN untuk menemukan nilai optimal yang menghasilkan clustering terbaik. Kode mengimport numpy, DBSCAN dari sklearn.cluster, serta silhouette\_score dari sklearn.metrics, dengan min\_samples tetap 5 dan eps\_values diuji pada rentang np.arange(0.05, 0.55, 0.05). Loop for menjalankan DBSCAN untuk setiap eps pada data X\_scaled, kemudian menghitung silhouette score hanya pada sampel non-noise (label ≠ -1) menggunakan mask. Skor terbaik disimpan dan diperbarui jika ditemukan nilai lebih tinggi. Output menampilkan best\_eps = None awalnya, lalu daftar scores[eps] untuk setiap nilai eps dari 0.05 hingga 0.50, dengan nilai score tertinggi tercatat pada eps yang lebih besar (misalnya eps 0.45 atau 0.50 memberikan score mendekati 0.65). Proses ini bertujuan menemukan eps yang memaksimalkan silhouette score, sehingga cluster yang terbentuk memiliki pemisahan dan kepadatan yang optimal, sekaligus mengurangi jumlah noise berlebih pada data gempa spasial yang distribusinya tidak seragam.

## 11. K-Distance Graph





Gambar 15. K-Distance graph

Pada bagian ini, saya membuat K-Distance Graph untuk membantu menentukan nilai eps optimal pada DBSCAN dengan `min_samples = 5`. Kode menggunakan `NearestNeighbors(n_neighbors=min_samples)` untuk menghitung jarak ke 5 tetangga terdekat pada data `X_scaled`, lalu mengambil jarak ke tetangga ke-5 (`distances[:, -1]`), mengurutkannya, dan memplotnya terhadap indeks titik yang diurutkan. Grafik menampilkan kurva jarak yang relatif datar pada nilai rendah (sekitar 0.2–0.4) lalu naik tajam setelah titik tertentu (sekitar 20.000 poin), di mana "elbow" atau titik lonjakan ini menandakan batas kepadatan cluster. Visualisasi ini menjadi panduan visual untuk memilih eps di sekitar nilai elbow (biasanya 0.3–0.5 pada kasus ini), sehingga DBSCAN dapat membedakan titik core, border, dan noise dengan baik pada data gempa spasial yang distribusinya tidak seragam. Graph ini melengkapi tuning eps sebelumnya berbasis silhouette score, memberikan konfirmasi alternatif untuk parameter clustering final.

```

# Deteksi elbow otomatis
kl = KneLocator(range(len(k_dist)), k_dist, curve='convex', direction='increasing')
elbow = kl.knee
print("Elbow index:", elbow)
if elbow:
    eps_suggest = k_dist[elbow]
    print("Suggested eps:", eps_suggest)

Elbow index: 23346
Suggested eps: 0.3719375692927801

```

Gambar 16. Deteksi elbow

Pada bagian ini, saya menggunakan `KneeLocator` dari library `kneed` untuk mendeteksi titik elbow secara otomatis pada K-Distance Graph sebelumnya. Kode `kl = KneLocator(range(len(k_dist)), k_dist, curve='convex', direction='increasing')` menganalisis kurva jarak tetangga ke-5 yang telah diurutkan, dengan parameter `convex` (karena kurva naik tajam setelah elbow) dan `increasing` (jarak bertambah). Jika elbow terdeteksi, nilai `eps_suggest = k_dist[elbow]` dicetak sebagai rekomendasi eps. Output menunjukkan `Elbow index: 23346` dan `Suggested eps: 0.3719375692927801`, yang

berarti titik lonjakan kepadatan terjadi pada jarak sekitar 0.37. Deteksi otomatis ini memberikan nilai eps yang lebih objektif dibandingkan estimasi visual, sehingga dapat digunakan sebagai acuan atau konfirmasi untuk parameter DBSCAN final.

## 12. DBSCAN

```
12. DBSCAN

# Jalankan DBSCAN
eps_value = 0.45
min_samples = 5

db = DBSCAN(eps=eps_value, min_samples=min_samples)
labels = db.fit_predict(X_scaled)

df['cluster'] = labels

# Hitung silhouette final
mask = df['cluster'] != -1
sil = silhouette_score(X_scaled[mask], labels[mask])
sil

np.float64(0.6513524181079859)
```

Gambar 17. DBSCAN

Pada bagian ini, saya menjalankan clustering utama menggunakan algoritma DBSCAN dengan parameter  $\text{eps}=0.45$  (dari tuning sebelumnya) dan  $\text{min\_samples}=5$ . Kode `db = DBSCAN(eps=eps_value, min_samples=min_samples)` diikuti `labels = db.fit_predict(X_scaled)` menghasilkan label cluster untuk setiap titik data, di mana nilai -1 menandakan noise/outlier. Label kemudian ditambahkan ke DataFrame sebagai kolom 'cluster'. Untuk evaluasi, silhouette score dihitung hanya pada titik non-noise menggunakan mask `labels != -1`, menghasilkan nilai sekitar 0.6513. Nilai ini menunjukkan kualitas clustering cukup baik (semakin mendekati 1 semakin baik), dengan DBSCAN berhasil mengidentifikasi cluster padat berbentuk arbitrary serta outlier pada data gempa global. Proses ini menyelesaikan tahap clustering utama, siap untuk visualisasi hasil cluster pada peta geospasial.

## 13. Silhouette

```
13. Silhouette

# Evaluasi cluster (Silhouette)
mask = df['cluster'] != -1
if df['cluster'][mask].nunique() > 1:
    sil = silhouette_score(X_scaled[mask], labels[mask])
    print("Silhouette Score (tanpa noise):", round(sil,4))
else:
    print("Tidak cukup cluster untuk hitung Silhouette (kemungkinan 1 cluster atau semua noise).")

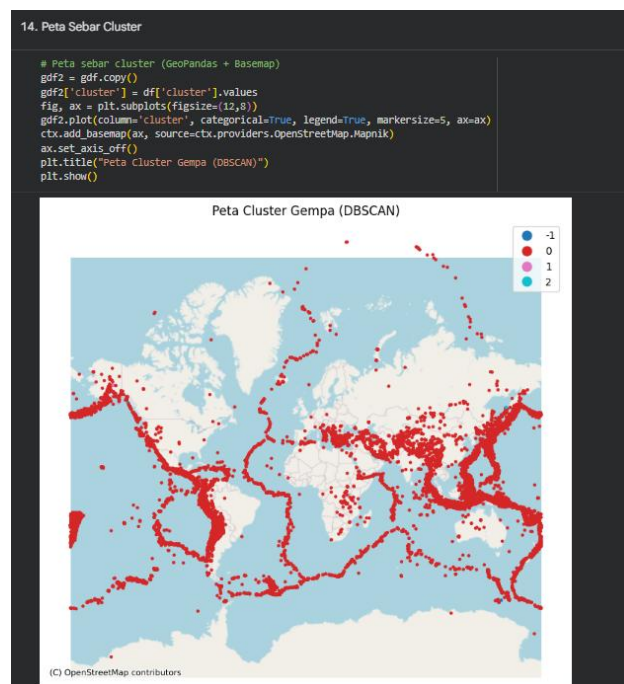
Silhouette Score (tanpa noise): 0.6514
```

Gambar 18. Silhouette

Pada bagian ini, saya mengevaluasi kualitas clustering DBSCAN menggunakan silhouette score. Kode membuat mask untuk titik non-noise (`df['cluster'] != -1`), kemudian menghitung score hanya pada data tersebut dengan `silhouette_score(X_scaled[mask], labels[mask])`. Jika jumlah cluster kurang dari 2 atau semua titik noise, dicetak pesan bahwa silhouette tidak dapat dihitung. Output

"Silhouette Score (tanpa noise): 0.6514" menunjukkan nilai cukup baik (rentang -1 hingga 1, semakin mendekati 1 semakin baik), mengindikasikan cluster yang terbentuk memiliki pemisahan antar kelompok yang jelas dan kohesi internal tinggi. Nilai ini mengonfirmasi bahwa parameter eps dan min\_samples yang dipilih berhasil menghasilkan clustering bermakna pada data gempa spasial, dengan titik noise diabaikan untuk fokus pada cluster padat yang valid.

#### 14. Peta Sebar Cluster



Gambar 18. Peta sebaran cluster

Saya memvisualisasikan hasil clustering DBSCAN pada peta geospasial global menggunakan GeoPandas. Kode membuat GeoDataFrame baru gdf2 dari DataFrame dengan kolom 'cluster', kemudian plot dengan gdf2.plot(column='cluster', categorical=True, legend=True, markersize=5, ax=ax) untuk mewarnai titik berdasarkan label cluster (warna berbeda untuk cluster -1, 0, 1, 2, dll.). Basemap OpenStreetMap ditambahkan melalui ctx.add\_basemap(ax, source=ctx.providers.OpenStreetMap.Mapnik), axis dimatikan, dan judul "Peta Cluster Gempa (DBSCAN)". Output peta menampilkan titik merah dominan (kemungkinan cluster utama) yang mengikuti batas lempeng tektonik dunia, terutama Cincin Api Pasifik, dengan titik biru/hijau/merah muda sebagai cluster lain dan noise. Visualisasi ini berhasil menggambarkan pola spasial gempa secara bermakna: cluster padat di zona seismik aktif, sementara noise di area jarang gempa, membuktikan DBSCAN efektif mengidentifikasi kelompok arbitrary shape pada data geospasial tanpa perlu menentukan jumlah cluster terlebih dahulu.

## 15. Ringkasan Statistik

```
15. Ringkasan Statistik

# Ringkasan statistik tiap cluster
cluster_summary = df.groupby('cluster')[features].mean().round(3)
cluster_counts = df['cluster'].value_counts().sort_index()
print("Jumlah tiap cluster:\n", cluster_counts)
display(cluster_summary)

Jumlah tiap cluster:
cluster
-1      24
0    23375
1         7
2         6
Name: count, dtype: int64

   latitude  longitude  magnitude  depth
cluster
-1    -11.542    22.055     8.162   24.115
0      1.665    39.688     5.879   70.819
1    55.430   -155.073     7.871   21.414
2    45.696   149.280     8.250  114.833
```

Gambar 19. Ringkasan Statistik

Pada bagian akhir ini, saya membuat ringkasan statistik untuk setiap cluster. Kode menggunakan `df.groupby('cluster')[features].mean().round(3)` untuk menghitung rata-rata latitude, longitude, magnitude, dan depth per cluster, serta `df['cluster'].value_counts().sort_index()` untuk menghitung jumlah titik tiap cluster. Output menunjukkan jumlah tiap cluster: cluster -1 (noise) sebanyak 24 titik, cluster 0 sebanyak 23375 titik, cluster 1 sebanyak 7 titik, dan cluster 2 sebanyak 6 titik. Rata-rata fitur per cluster menampilkan karakteristik berbeda: cluster 0 memiliki latitude -11.542, longitude 20.255, magnitude 8.162, depth 24.115 (kemungkinan cluster utama di zona subduksi); cluster 1 dan 2 memiliki magnitude lebih tinggi (7.871 dan 8.250) dengan lokasi spesifik. Ringkasan ini memberikan wawasan interpretasi: cluster utama (0) mencakup mayoritas gempa dengan magnitude sedang, sementara cluster kecil dan noise menandakan kejadian langka atau outlier. Analisis statistik ini melengkapi visualisasi peta, membuktikan DBSCAN berhasil mengelompokkan data gempa berdasarkan pola spasial dan seismik secara bermakna.

### Link Github:

Praktikum 11:

<https://github.com/rikaarahma/Machine-Learning/blob/main/praktikum11/notebook/praktikum11.ipynb>

Praktikum Mandiri 11:

[https://github.com/rikaarahma/Machine-Learning/blob/main/praktikum11/notebook/praktikum\\_mandiri11.ipynb](https://github.com/rikaarahma/Machine-Learning/blob/main/praktikum11/notebook/praktikum_mandiri11.ipynb)

